

Este trabalho implementa uma versão simplificada do jogo Donkey Kong a partir da biblioteca gráfica Raylib. Ele possui duas telas principais: o menu, e a tela de jogo. O menu e toda a interface gráfica do jogo estão contidos na biblioteca `interface.h` e implementadas na `interface.c`.

A função principal do menu (`int Menu(void)`) e do menu de pausa (`int MenuPausa(void)`) retornam um inteiro que, no loop principal do jogo, é verificado para inicializar o jogo ou voltar ao jogo, ver o score, ou voltar ao menu principal ou sair. Ela chama as funções:

- `void InitBotao(Botao* btn, const char* caminhoTextura, Vector2 pos, float scale);`
- `void UnloadBotao(Botao* btn);`
- `void desenhaBotao(Botao* btn);`
- `bool botaoPressionado(Vector2 mousePos, Botao* btn, bool mousePressionado);`

Que, respectivamente, inicializam na memória determinado botão, desalocam o local da memória, desenharam os botões com suas determinadas texturas (todas as texturas encontram-se na pasta 'graficos' e são definidas em uma `struct` dentro de '`interface.h`').



(a) Interface do Menu Principal



(b) Interface do Menu de Pausa

Figura 1: Menus.

A implementação do jogo começa a partir dos arquivos `mapasX.txt`, nos quais `X` representa a fase. Eles contêm os seguintes caracteres identificadores:

Z: Chão

D: Descida de uma escada

H: Escada

S: Início de uma escada

M: Monstro

E: Especial

P: Personagem

L: Final (utilizado no mapa final)

O desenho do cenário ocorre a partir das funções apresentadas na subseção **FUNÇÕES PARA DESENHAR O MAPA** no arquivo `interface.c`:

- `void carregaMapa(const char* caminhoArquivo, char m[MAPA_ALTURA][MAPA_LARGURA], Boneco *bombeiro);`
Lê o arquivo `mapasX.txt` correspondente e gera uma matriz de 30×30 elementos na memória para que ela possa ser lida e interpretada.
- `void desenhaMapa(TexturasJogo text, char m[MAPA_ALTURA][MAPA_LARGURA], Boneco *bombeiro);`
Desenha na tela, a cada iteração do loop do jogo, todos os elementos com base na matriz carregada e na posição atual do personagem (indicada na sua própria `struct`).
- `void desenhaCoracao(TexturasJogo text, Boneco bombeiro);`
Desenha os corações no canto superior esquerdo da tela.
- `void reiniciaFase(char m[MAPA_ALTURA][MAPA_LARGURA], Boneco *bombeiro, int faseAtual);`
Recomeça a fase atual e redefine as posições quando o personagem atinge um monstro.

Todo o cenário é construído a partir de texturas contidas na pasta `gráficos` e declaradas na `struct Texturas`.

```

1 .....
2 .....D.....F.....
3 ...ZZZZHZZZZZZZZZZZZZZZZZZZZ...
4 .....H.....
5 .....S.D.....M.D.....
6 ...ZZZZZZZZZZZZZZZZZZHZZZZZZ...
7 .....H.....
8 .....H.....
9 .....H.....
10 .....D.....M..S.....
11 ...ZZZZZHZZZZZZZZZZZZZZZZZZ...
12 .....H.....
13 .....S.....M.....D.....
14 ZZZZZZZZZZZZZZZZZZZZZZZZZZZHZZZZZZ
15 .....H.....
16 .....H.....
17 .....H.....
18 .....D..M.....S.....
19 .....ZZZHZZZZZZZZZZZZZZZZZZZZ
20 .....H.....
21 .....H.....
22 ...M.....S.....D.E.....
23 ...ZZZZZZZZZZZZZZZZZZHZZZZ...
24 .....H.....
25 .....H.....
26 .....P.....S.....
27 ZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ
28 .....
29 .....
30 .....

```

(a) Mapa de uma fase genérica

```

1 .....
2 .....D.....L.....
3 ...ZZHZZZZZZZZZZZZZZZZZZZZZZZZZZ...
4 .....H.....
5 .....H.....
6 .D...S..M...D.....
7 ZHZZZZZZZZZZZZH.....
8 .H.....H.....
9 .H.....S.....DM.....
10 .H....ZZZZZZZZZZZZZZHZZZZZZ...
11 .H.....H.....
12 .S..D.M.....D...S..M.....
13 .ZZZHZZZ...ZZZZHZZZZZZZZZZZZ...
14 ...H.....H.....
15 ...H.....H.....
16 ...S.....M...S.....D.....
17 ...ZZZZZZZZZZZZZZZZZZZZHZZ...
18 .....H.....
19 .....H.....
20 .....E..M....D.....S....
21 ...ZZZZZZZZZZHZZZZZZZZZZZZZZZZ...
22 .....H.....
23 .....H.....
24 .....S.....D..M.....
25 .....ZZZZZZZZZZZZZZHZZZZZZ...
26 .....H.....
27 .....H.....
28 ..P.....S.....
29 ZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZZ
30 .....

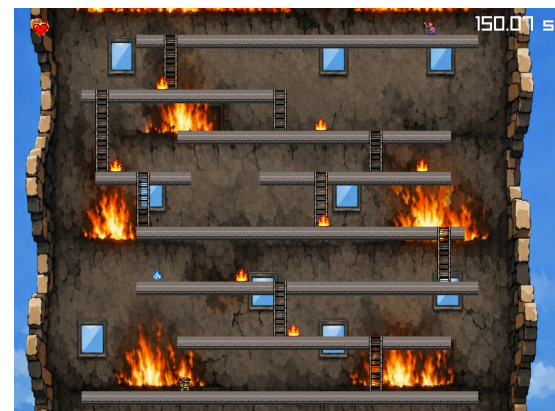
```

(b) Mapa Final

Figura 2: Estrutura do mapaX.txt



(a) Tela de uma fase genérica(porta)



(b) Tela da fase final(menina)

Figura 3: Telas do jogo.

A movimentação do personagem e dos monstros acontece nos arquivos `movimentacao.h` e `movimentacao.c`, na qual a `int movimentoPersonagem(char m[MAPA_ALTURA][MAPA_LARGURA], Boneco *bombeiro)`; retorna um inteiro representando se o movimento foi bem sucedido (0) ou se passou de fase (1); todo o movimento é a partir da verificação da matriz do mapa pelas posições x e y do personagem, verificando os limites do mapa, se o personagem pode subir ou descer escadas. A movimentação do pulo é calculada pelo tempo, cada pulo dura .35 segundos e possui um atraso de .15 segundos, permitindo que o personagem ande no ar sem parecer que ele voa.

Para o monstro, há cinco funções:

`void initMonstro(Monstro monstros[], int* numMonstros, char m[MAPA_ALTURA][MAPA_LARGURA], float blocoTamanho)`; que inicializa os monstros no mapa percorrendo a matriz e definindo suas propriedades ao encontrar um 'M';

`void movimentoMonstro(Monstro* monstro, char m[MAPA_ALTURA][MAPA_LARGURA], float blocoTamanho)` que adiciona delay à movimentação do monstro e depois tenta mover na direção atual checando se o bloco à frente tem chão abaixo ('Z' ou 'H'). Se não tiver, inverte a direção;

`int verificaColisaoMonstro(char m[MAPA_ALTURA][MAPA_LARGURA], Monstro* monstro, float blocoTamanho, Boneco *b)`; verifica se o monstro está na mesma célula do personagem. Se sim e o personagem não estiver em modo especial (`especial == false`), retorna 1 indicando colisão;

`void desenhaMonstro(Monstro* monstro, TexturasJogo texturas, float blocoTamanho, float comecoMapa)`; que desenha o monstro na tela usando a textura dele, utilizando o `DrawTexturePro` para que as dimensões do boneco condizam com o mapa;

`int atualizaMonstro(Monstro* monstro, char m[MAPA_ALTURA][MAPA_LARGURA], TexturasJogo texturas, Boneco *b)`; que é a função principal utilizada no loop do jogo para atualizar o estado do monstro a cada frame, chamando as outras funções para calcular o tamanho do bloco, mover o monstro, desenhá-lo na tela e verificar colisão com o personagem

O main

O main, `jogo.c` carrega todas as texturas do jogo e inicializa as variáveis de controle do jogo como vidas, fase atual, placar e tempo. Abre o arquivo "placar.bin" para leitura e escrita, inicializa o vetor de placares e lê os dados existentes do arquivo.

No loop principal (`while !WindowShouldClose`), exibe o menu e aguarda a escolha do jogador. Se voltou da pausa, pula direto pro topo sem chamar o menu de novo. Se o retorno for 2, encerra o jogo.

No caso 0 (jogar), reinicia todas as variáveis, carrega o Mapa 1 e inicializa os monstros. Entra no loop do jogo onde a cada frame: move o personagem, verifica se passou

de fase (carregando o próximo mapa até o 3), verifica pausa (que pode voltar ao jogo, ao menu ou fechar), e então desenha tudo — fundo, mapa, tempo decorrido, corações e monstros. Se um monstro colidir com o personagem, verifica as vidas: se for a última exibe a tela de derrota e o placar; caso contrário decrementa a vida, adiciona 3 segundos de penalidade de tempo e reinicia a fase. Ao vencer o mapa 3, exibe a tela de vitória e oferece registrar o placar ou visualizá-lo, dependendo do resultado do jogo.

Ao sair do loop principal, descarrega todas as texturas, fecha o arquivo e encerra a janela.

Outras funções utilizadas no main:

- `void RegistraPlacar(TIPO_PLACAR placar[], float tempo, FILE *arq);`
Registra o placar se necessário.
- `void desenhaPlacarNaTela(TIPO_PLACAR placar[]);`
Desenha o placar na tela.
- `int desenhaVitoria(void);`
Desenha a tela de vitória.
- `int desenhaDerrota(void);`
Desenha a tela de derrota.

Mecânicas importantes:

- **W / Seta ↑:** personagem sobe escada
- **S / Seta ↓:** personagem desce escada
- **D / Seta →:** personagem vai para direita
- **A / Seta ←:** personagem vai para esquerda
- **Espaço:** personagem pula
- **TAB / P:** pausa no jogo
- **Personagem encostar na água:** invencibilidade por 5 segundos
- **Personagem tocar no monstro:** perde uma vida e adiciona 3 segundos ao tempo total

COMO RODAR O JOGO:

No .zip há um arquivo .exe que roda automaticamente o jogo, ou, se necessário, abrir o arquivo no 'vscode' e digitar no terminal:

```
gcc *.c -o jogo.exe -IC:/raylib-6.0_win64_mingw-w64/include -LC:/raylib-6.0_win64_mingw-w64/lib -lraylib -lopengl32 -lgdi32 -lwinmm
```

Depois:

```
.\jogo.exe
```